Custom Web Scraper

A Deep Dive, from zero prior knowledge

Portfolio explainer, built from the source

The one-sentence version

`custom-web-scraper` is a desktop application that opens a real Chrome browser, points it at a web address you type, and then lets you ask that page questions like "show me every element whose class is `product-title`", answering in a table instead of making you write a fresh Python script each time.

This document assumes you know nothing about web scraping, browsers, or Python graphical applications. Every term is defined the first time it appears. It covers the whole project: all 551 lines of it, in one file, plus its configuration and its dependency. It is honest about what the code does well and what it does badly, because it exists to prepare its owner to defend the project in an interview, not to flatter it.

Contents

1 What problem does this solve?

1.1 Starting from the very beginning

Jargon: Web page, HTML and the DOM

A web page arrives at your computer as **HTML**: a text document made of nested tags such as `<p>Hello</p>` or `<a href="/next">Next page</a>`. The browser parses that text into a live tree of objects in memory called the **DOM** (Document Object Model). "The DOM" is simply the browser's in-memory picture of the page. When people say a page "changed without reloading", they mean a program edited the DOM.

Jargon: Web scraping

Scraping means extracting data out of a web page automatically, rather than reading it with your eyes. If you wanted the price of every item in an online shop, scraping is writing a program that finds each price element and reads its text.

Jargon: Element and locator

An **element** is one node of the DOM tree: one link, one paragraph, one image. A **locator** is a rule for finding elements, for example "every element whose class attribute is `price`". A locator has two halves: a *strategy* (search by class? by ID? by tag?) and a *value* (which class? which ID?). That two-part split is the single most important idea in this whole application, and it is exactly what its user interface exposes.

1.2 The workflow this replaces

Suppose you want to poke at an unfamiliar page: what links are on it, what is inside a particular section, does this XPath actually match what you think it matches. The traditional answer is to write a small throwaway Python script, run it, read the output, edit the script, run it again. Each question costs an edit-run cycle, and each new site costs a whole new script.

This project's premise, stated plainly in its README, is that those small investigations do not need a script; they need a few minutes of poking. So it turns the loop inside out: load the page once into a browser that stays open, then fire locator after locator at that same live page from a graphical interface, with no code involved and no reload between attempts.

Why a real browser, and not a simple download?

The naive way to scrape is to download the page's HTML text over the network and search it. That fails on any **JavaScript-rendered** page, which means most modern sites. The first HTML response for such a page is nearly empty; the actual content is created afterwards by code that runs *inside the browser*. If you never run a browser, you never see that content. Driving real Chrome means the page is fully built, exactly as a human would see it, before this tool reads anything. The cost of that choice is that you must have Chrome installed and a window opens on your screen.

Jargon: Selenium and WebDriver

Selenium is a Python library for remote-controlling a real browser. It speaks **WebDriver**, a standard protocol that browsers implement, via a small helper program (for Chrome, `chromedriver`). Your Python code says "navigate here", "find these elements", and the browser obeys. The `driver` object in this project is the handle on that browser session.

Jargon: Tkinter

Tkinter is the graphical user interface toolkit that ships inside Python itself. It gives you windows, buttons, text fields and tables without installing anything. `ttk` is its themed widget set, a more modern-looking layer over the same toolkit. This project mixes both, a point returned to in Section 14.

2 The shape of the project on disk

The entire application is one Python file. There is no package, no module tree, no test directory.

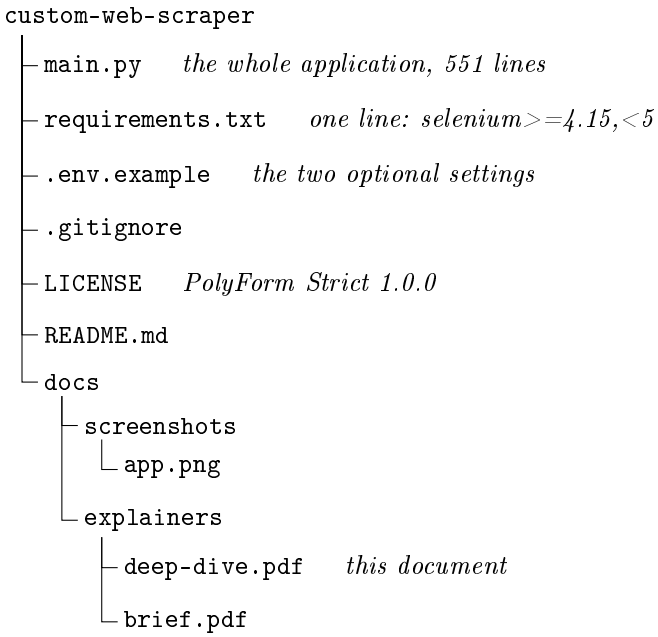

Figure 1: Every tracked file. The application is `main.py` and nothing else.

One file is a defensible choice here

A single 551-line script for a single-window tool is not a flaw by itself. Splitting it into six modules would add navigation cost without removing any complexity. What *is* a flaw is how that file shares its state, which Section 12 covers in detail.

3 Architecture: three layers, one file

The code separates into three layers. They are not marked by folders or classes, only by convention and comment banners inside `main.py`, but the separation is real and it is the most interesting design decision in the project.

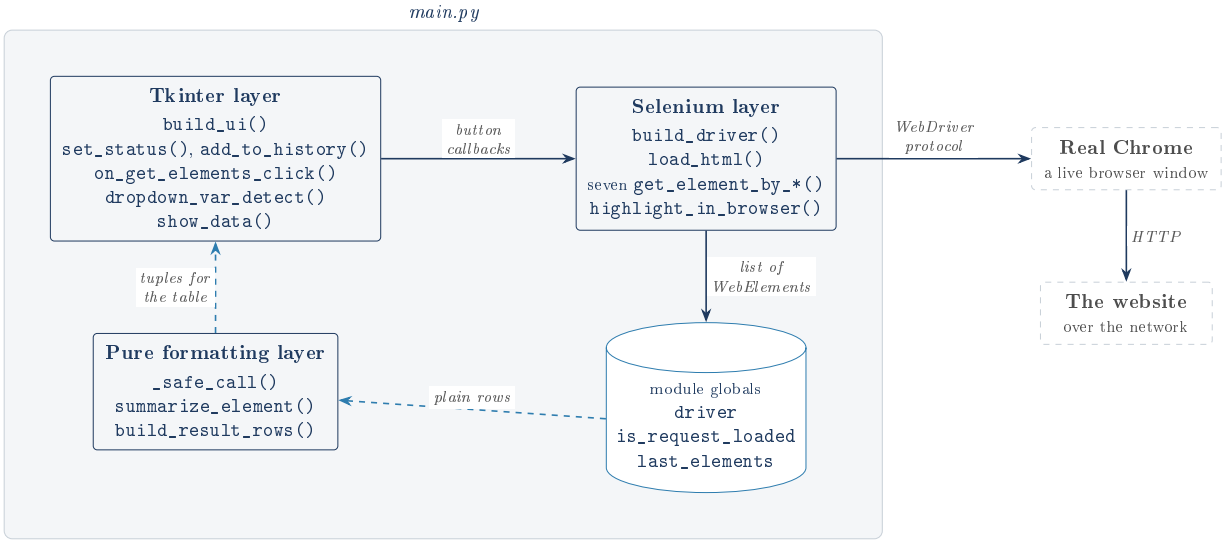

Figure 2: The three layers, the state they share, and the two things outside the process. Solid arrows are commands going out; dashed arrows are display data coming back.

3.1 Why the third layer exists

The **pure formatting layer** is the part worth defending in an interview. Its job is to turn Selenium's live element objects into flat tuples the table can show. It is called *pure* because it never touches `driver`, reads no global, and only calls read-only methods on whatever object it is handed.

Duck typing is what makes this testable

Duck typing is the Python principle that an object's type does not matter, only whether it has the methods you call ("if it walks like a duck" ...). `summarize_element` only ever reads `.tag_name`, `.text` and `.get_attribute(...)`. So a five-line fake class with those three members is indistinguishable from a real Selenium element as far as this function can tell. That means the formatting logic can be tested in milliseconds with no browser, no network and no window. The rest of the file cannot make that claim.

4 Startup: what happens when you run it

```
if __name__ == "__main__":
    driver = build_driver()
    build_ui()
    root.mainloop()
    driver.quit()
```

Listing 1: The entry point, the last five lines of main.py

Jargon: The `__name__ == "__main__"` guard

Python sets the variable `__name__` to the string `"__main__"` when a file is run directly, but to the module's name when it is imported by another file. So this block runs only on `python main.py`, and is skipped on `import main`. That is what lets a test import the file and call individual functions without launching Chrome.

Jargon: The event loop (`mainloop`)

A graphical program does not run top to bottom and exit. It enters an **event loop**: an endless cycle of "wait for the user to do something, find the function registered for that something, call it, repeat". Those registered functions are **callbacks**. `root.mainloop()` is that cycle, and it does not return until the window is closed, which is why `driver.quit()` sits after it.

4.1 `build_driver()`: getting a browser

```
chrome_options = Options()
if os.environ.get("SCRAPER_HEADLESS", "").lower() in ("1", "true", "yes"):
    chrome_options.add_argument("--headless=new")

chromedriver_path = os.environ.get("CHROMEDRIVER_PATH", "").strip()
service = Service(chromedriver_path) if chromedriver_path else Service()

new_driver = webdriver.Chrome(service=service, options=chrome_options)
new_driver.maximize_window()
return new_driver
```

Listing 2: Driver construction, simplified to its decisions

Two decisions live here, both driven by **environment variables**.

Jargon: Environment variable

A named value the operating system hands to a program when it starts, set outside the code (`SCRAPER_HEADLESS=1 python main.py`). It is the standard way to configure a program per-machine without editing its source, which matters here because the two settings differ between the developer's laptop and any other.

- `SCRAPER_HEADLESS`, if set to `1`, `true` or `yes`, adds the `--headless=new` flag, so Chrome runs with no visible window. Note the accepted values are checked after `.lower()`, so `TRUE` works and `yes please` does not.
- `CHROMEDRIVER_PATH`, if set, points at a specific `chromedriver` binary. If it is empty, `Service()` is constructed with no argument and **Selenium Manager** takes over.

Selenium Manager, and the portability bug it fixed

Historically you had to download a `chromedriver` whose version matched your installed Chrome exactly, and re-download it every time Chrome auto-updated. Selenium 4.6 and later ship **Selenium Manager**, which detects your Chrome and fetches the right driver by itself. The README records that this project's original version hardcoded a Windows path, `C:\Development\chromedriver.exe`, so it ran on exactly one machine on earth. The environment-variable-with-fallback pattern here is the fix, and it is a genuinely good answer to "tell me about a portability problem you solved".

`maximize_window()` is called even when headless

`build_driver` calls `new_driver.maximize_window()` unconditionally, including in the headless path where there is no window on screen to maximize. In practice modern headless Chrome tolerates this and simply sets a large viewport, so it is harmless rather than broken, but it reads as an oversight: the call was written for the visible case and never revisited for the headless one. Flagged as *inferred* intent; the code carries no comment either way.

5 Loading a page: load_html()

```
def load_html():
    global is_request_loaded
    url = request_url_entry.get().strip()
    set_status("Loading page...", "info")
    root.update_idletasks()
    try:
        driver.get(url)
        is_request_loaded = True
    except InvalidArgumentException:
        set_status("Please enter a valid URL", "error")
        return

    try:
        WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.CSS_SELECTOR, "html
    ")))
        WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.CSS_SELECTOR, "body
    ")))
    except TimeoutException:
        set_status("Network error: the page did not load in time", "error")
        return

    add_to_history(request_url_entry, url)
    set_status(f"Loaded {url}", "success")
```

Listing 3: load_html, in full

5.1 The update_idletasks call

`set_status("Loading page...")` changes a label's text, but Tkinter does not repaint the screen until the event loop next gets a turn, and the loop is about to be blocked for seconds inside `driver.get`. Without `root.update_idletasks()`, which forces the pending redraw to happen immediately, the user would never see "Loading page..."; the window would simply freeze and then show the finished state. It is one line, easy to skim past, and it is the difference between an app that looks broken and one that does not.

5.2 Waiting properly

Jargon: WebDriverWait and expected conditions

A page loads over an unpredictable amount of time. `WebDriverWait(driver, 10).until(condition)` polls repeatedly for up to 10 seconds until `condition` is true, then returns instantly. If the 10 seconds elapse first, it raises `TimeoutException`. The condition used here, `presence_of_element_located`, means "an element matching this locator now exists in the DOM". The alternative, `time.sleep(5)`, is both slower than necessary when the page is fast and flaky when it is slow. This is the correct tool and the project uses it consistently.

The function waits for `<html>` and then `<body>` to exist. Both are present in essentially any successful page load, so this is a broad "did we get a document at all" check rather than a "is the content ready" one.

Two real defects in this function

1. **`is_request_loaded` is set too early.** It becomes `True` the moment `driver.get` returns, *before* the `<html>`/`<body>` wait. If that wait then times out, the function reports a network error and returns, but the flag is left `True`. The application now believes a page is loaded

when it is not, and every subsequent locator lookup will sail past its guard and fail in a more confusing way. The flag should be set on the last line, next to the success message.

2. **Only one exception type is caught.** `InvalidArgumentException` covers a malformed URL such as `notaurl`. But `driver.get` can also raise `WebDriverException` for a host that does not resolve, a refused connection, or a browser that has crashed. None of those are caught, so they escape the callback and print a raw traceback to the terminal while the GUI just sits there, status bar still reading "Loading page…". For a tool whose selling point is that it never fails silently, this is the gap.

6 The seven locator functions

This is the heart of the tool, and also its most repetitive code. There are seven functions, one per Selenium locator strategy.

Dropdown label	Function	Selenium strategy
Class	`get_element_by_class_name`	`By.CLASS_NAME`
ID	`get_element_by_id`	`By.ID`
Partial Link Text	`get_element_by_partial_link_text`	`By.PARTIAL_LINK_TEXT`
Link Text	`get_element_by_link_text`	`By.LINK_TEXT`
Name	`get_element_by_name`	`By.NAME`
Tag Name	`get_element_by_tag_name`	`By.TAG_NAME`
XPath	`get_element_by_xpath`	`By.XPATH`

Table 1: `LOCATOR_DISPATCH`: the seven strategies the GUI offers, and the Selenium constant each one maps to.

Jargon: The attributes these search on

`id` is meant to be unique on a page; `class` is a shared style label and usually matches many elements; `name` is used by form fields; a **tag name** is the element type itself (`a`, `div`, `p`). **Link text** matches an `<a>` element by the words a user actually sees, and the partial variant matches a substring of them. **XPath** is a small query language for walking the document tree, for example `//div[@class='x']/a`, and it is the escape hatch that can express anything the other six cannot.

Every one of the seven has the identical shape:

```
def get_element_by_class_name(class_name):
    global is_request_loaded
    if not is_request_loaded:
        set_status("Please request a URL first", "error")
        return None, None
    try:
        WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.CLASS_NAME,
    class_name)))
        value = driver.find_elements(By.CLASS_NAME, class_name)
    except TimeoutException:
        set_status("Please enter a valid class name", "error")
        return None, None
    return True, value
```

Listing 4: get_element_by_class_name; the other six differ only in the By constant and the error string

6.1 Wait, then find: why both calls?

The pairing is deliberate. A `WebDriverWait` on `presence_of_element_located` blocks until *at least one* match exists, which handles the case where JavaScript has not yet inserted the element. Then `find_elements` (plural) fetches *all* of them. Using `find_elements` alone would return an empty list immediately on a page still rendering; using the wait alone would return only the first match.

Jargon: The `(condition, data)` return convention

Each function returns a two-item tuple. On success: `(True, [elements])`. On any failure: `(None, None)`, having already displayed its own error message. The caller therefore only has to check the first item. It is a hand-rolled version of what other languages express with a `Result` type. It works, but returning `None` rather than `False` for the failure flag is loose: the caller tests `if not condition`, which treats both the same, so nothing breaks, but the type is inconsistent with the `True` on the success path.

Seven copies of the same twelve lines

The only differences between these functions are the `By` constant and the wording of one error string. The `is_request_loaded` guard, the `WebDriverWait`, the `TimeoutException` handler and the return shape are duplicated seven times. A single function taking the strategy as an argument, with the labels in a table, would collapse all seven into roughly fifteen lines total. The README acknowledges this directly and honestly: a fix in one has to be hand-copied to the other six.

Worth noting what the project *did* fix. `LOCATOR_DISPATCH` replaced an `if`/`elif` chain in the click handler, so adding an eighth strategy no longer means editing the dispatch logic. That is the right instinct applied one level too shallowly: the table removed the duplication in the *caller* but left it in the *callees*.

The zero-match branches are unreachable (verified, not guessed)

`on_get_elements_click` carefully handles a count of zero: it picks the `"info"` status color rather than `"success"`, disables the highlight button, and `show_data` renders a “No elements matched this locator.” label when there are no rows.

None of that can ever run. Reaching `find_elements` at all requires the `WebDriverWait` on the same locator to have already succeeded, which means at least one match exists. If nothing matches, the wait raises `TimeoutException` and the function returns `(None, None)`, so the caller returns before any of the zero-handling code. The only way to see a count of zero is a page mutating in the milliseconds between the wait and the find.

This was confirmed by driving the real code with a fake driver and a stubbed `WebDriverWait`: with the wait forced to succeed, a zero-element result does produce “Found 0 elements” and a disabled button, proving the branch is written correctly. It is simply not reachable through the front door.

The user-visible consequence is a mislabelled error. A locator that is perfectly valid but happens to match nothing reports “Please enter a valid class name”, blaming the user for a typo they did not make. The honest message would be “no elements matched”, and the code to display it already exists.

7 The click handler and the full data flow

```
def on_get_elements_click():
    global is_request_loaded
    if not is_request_loaded:
```

```
        set_status("Please request a URL first", "error")
        return

    locator_type = dropdown_var.get()
    lookup = LOCATOR_DISPATCH.get(locator_type)
    if lookup is None:
        set_status("Pick a locator strategy first", "error")
        return

    locator_value = get_element_by_entry.get().strip()
    condition, data = lookup(locator_value)
    if not condition:
        return

    add_to_history(get_element_by_entry, locator_value)
    count = len(data)
    set_status(f"Found {count} element{'s' if count != 1 else ''}", "success" if count else "
    info")
    highlight_button.config(state="normal" if count else "disabled")
    show_data(data)
```

Listing 5: on_get_elements_click, the function that ties the layers together

The dropdown starts out holding the string `"Select"`. That string is deliberately *not* a key in the dispatch table, so `.get()` returns `None` and the user is told to pick a strategy first. Using the dictionary's own miss as the validation check is neat: there is no second list of valid strategies to keep in sync with the first.

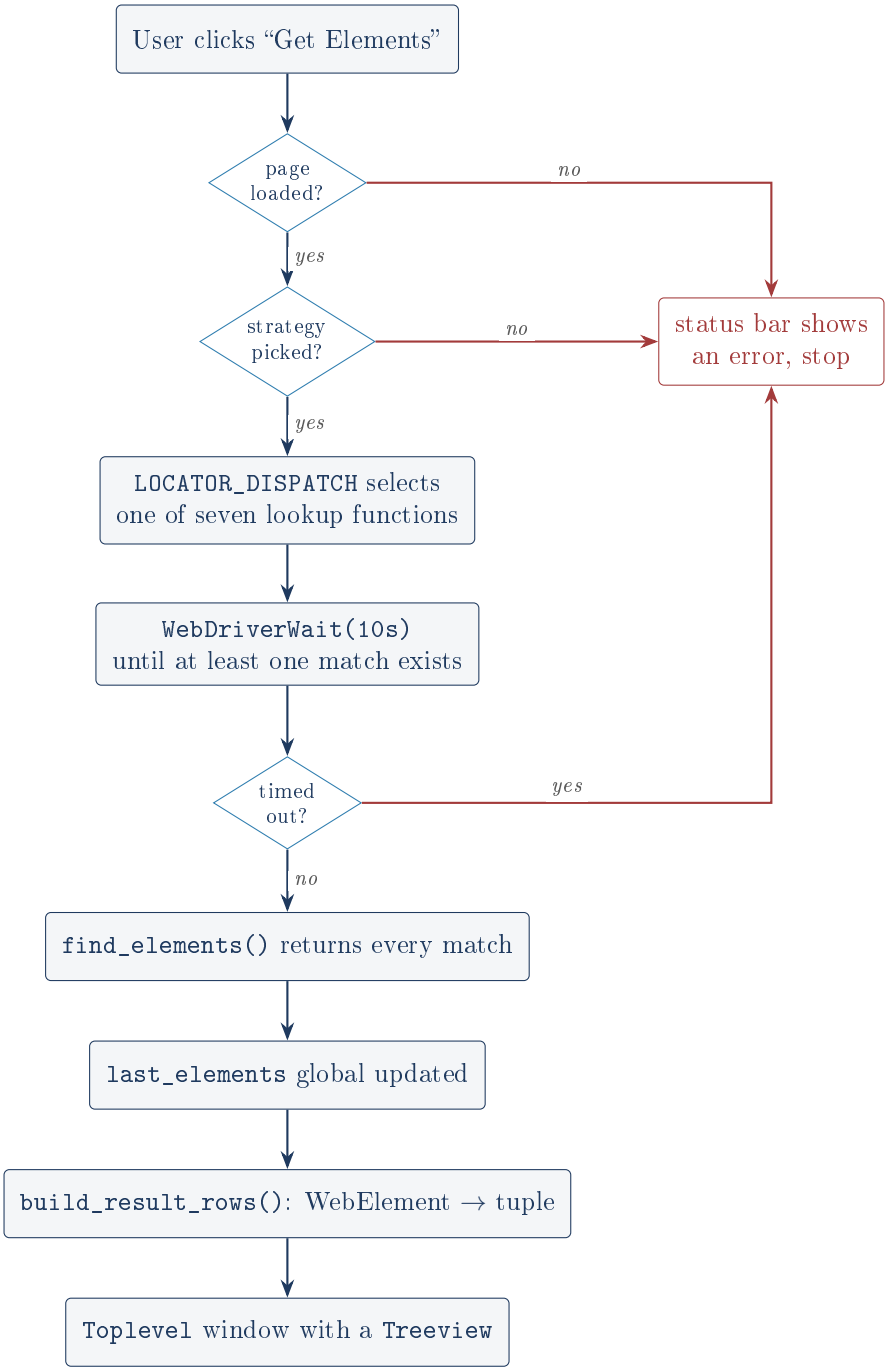

Figure 3: The full path of one lookup. Note that the “timed out” branch is the one that fires when a locator simply matches nothing, which is why its error message is misleading.

8 The pure formatting layer

```
ATTRIBUTES_TO_SHOW = ("id", "class", "name", "href", "src", "type", "value")

def _safe_call(fn):
    try:
        return fn()
    except Exception:
        return None

def summarize_element(element, index, text_limit=80):
    tag = _safe_call(lambda: element.tag_name) or "?"
```

```
11
12      text = _safe_call(lambda: element.text) or ""
13      text = " ".join(text.split())
14      if len(text) > text_limit:
15          text = text[: text_limit - 1] + "..."
16
17      attr_parts = []
18      for attr in ATTRIBUTES_TO_SHOW:
19          value = _safe_call(lambda a=attr: element.get_attribute(a))
20          if value:
21              attr_parts.append(f"{attr}={value}")
22      attributes = ", ".join(attr_parts)
23
24      return index, tag, text, attributes
25
26  def build_result_rows(elements, text_limit=80):
27      return [summarize_element(el, i + 1, text_limit=text_limit) for i, el in enumerate(
        elements)]
```

Listing 6: The formatting layer, in full

8.1 `_safe_call` and the stale element problem

Jargon: Stale element reference

A Selenium element object is a *handle* pointing into the live browser, not a copy of the data. If the page re-renders between the moment you find an element and the moment you read it, that handle refers to something that no longer exists, and any access raises `StaleElementReferenceException`. This is not an edge case on a modern site; it is routine.

`_safe_call` takes a zero-argument function, calls it, and returns `None` instead of propagating any exception. The granularity is the point: it wraps *each individual field access*, not the whole row. So an element that goes stale halfway through still produces a row with whatever fields were read successfully, rather than blanking the entire table.

The `lambda a=attr:` trick is load-bearing

Inside the attribute loop the lambda is written `lambda a=attr: element.get_attribute(a)`, not `lambda: element.get_attribute(attr)`. Python closures capture *variables*, not values, so a lambda written the second way would look up `attr` when it is finally called and see whatever the loop variable holds at that moment. Binding it as a default argument (`a=attr`) captures the value at definition time. Here the lambda is called immediately so both forms would in fact work, but the defensive form is the correct habit and costs nothing.

8.2 Verified behaviour

Because this layer is pure, its claims can simply be checked. Running the real functions against fake duck-typed objects confirmed all of the following:

- Whitespace is collapsed: a string with a newline and doubled spaces in it comes back as single-spaced. The idiom responsible is `" ".join(text.split())`, which splits on any run of whitespace and rejoins with single spaces.
- Truncation is exact: 200 characters of input produce a text field of *exactly* 80 characters, being 79 characters plus a single-character ellipsis. The `text_limit - 1` is there precisely so the ellipsis fits inside the budget rather than pushing past it. This is the kind of off-by-one that is usually wrong; here it is right.

- An element that raises on every single field access yields `(3, '?', '', '')`. The `or "?"` fallback fires for the tag, and the row survives.
- Attributes not in `ATTRIBUTES_TO_SHOW` are dropped, and attributes whose value is empty are skipped by the `if value:` test, so you never see a useless `class=` in the summary.
- `build_result_rows` numbers rows from 1, not 0, because it is display data for a human.

9 The results window: `show_data()`

Jargon: `Toplevel` and `Treeview`

A `Toplevel` is a second, independent window owned by the main one. A `ttk.Treeview` configured with `show="headings"` is a multi-column table: a spreadsheet-like grid with a header row. It is the standard Tkinter answer to "display a list of records".

`show_data` first stores `last_elements = list(data_to_show)`, taking a copy so the highlight button has something to work with later, then builds the rows, then constructs a fresh window with four columns: `index`, `tag`, `text`, `attributes`. Two scrollbars are attached, vertical and horizontal, wired to the tree with the standard `yscrollcommand`/`command` cross-reference.

The layout inside that window is worth noticing because it is done correctly. Two calls on the frame, `rowconfigure(0, weight=1)` and `columnconfigure(0, weight=1)`, tell the grid that the table, and only the table, absorbs extra space when the window is resized. Without them the table would stay a fixed size inside a growing window. Setting `selectmode="browse"` limits selection to one row at a time.

Every lookup opens another window, and nothing closes them

`show_data` constructs a new `Toplevel` on every single click of "Get Elements". It never reuses or closes the previous one. Poke at a page ten times, which is precisely the workflow the README describes as the reason the tool exists, and you have ten stacked result windows to close by hand. Reusing one window, or at least destroying the previous, would be a small change. As written, the tool's core use case is also the thing that clutters the screen fastest.

The results table is not sortable, though the README says it is

The README's opening paragraph states results "land in a sortable table". They do not. `ttk.Treeview` does not sort by default; making a column sortable requires binding a callback to the heading via `tree.heading(col, command=...)` and re-inserting rows in order. No such binding exists in `main.py`. The table is scrollable and selectable, but clicking a column header does nothing at all. Where the README and the code disagree, the code wins, and this is a claim to remove from the README rather than defend.

10 Highlighting: reaching into the live page

```
driver.execute_script(
    "arguments[0].style.outline='3px solid #ff3b81';"
    "arguments[0].style.outlineOffset='2px';"
    "arguments[0].scrollIntoView({block: 'center'});",
    element,
)
```

Listing 7: The JavaScript injected into the real browser

Jargon: `execute_script`

Selenium can run arbitrary JavaScript inside the page being viewed. Any Python elements passed after the script string arrive in JavaScript as the `arguments` array, so `arguments[0]` *is* the real DOM node corresponding to the Python element handle. This is the bridge between the two languages, and it is why the outline appears in the actual Chrome window rather than in a picture of it.

The script does three things per element: draws a pink outline, offsets that outline slightly so it does not sit flush against the content, and scrolls the element to the centre of the viewport. The loop counts successes and wraps each in `try`/`except Exception: continue`, so stale elements are skipped rather than aborting the run. If every element failed, `highlighted` stays 0 and the status honestly reports that the page may have changed.

This is the most genuinely useful feature in the tool. It closes the loop between "my locator matched 12 things" and "are they the 12 things I meant?", which is the question a locator-testing tool exists to answer.

Highlighting can point at a page you already left

`last_elements` and the highlight button's enabled state are only ever updated on a *successful* lookup. Load page A, find elements, then load page B: `last_elements` still holds A's handles and the button is still enabled. Clicking it now walks handles into a document that is gone. In practice every access raises, the loop skips all of them, and the user sees "Could not highlight those elements (page may have changed)", which is a truthful message arrived at by luck rather than design. `load_html` should clear `last_elements` and disable the button; it does neither.

The same staleness applies after a *failed* lookup: the function returns early without touching either, so the previous result stays live and highlightable behind a fresh error message.

11 The interface layer

11.1 `build_ui()` and why it is a function

`build_ui` constructs every widget and returns `root`. The README is explicit that it was split out from the `__main__` block on purpose, so that a test can build the entire window against a fake driver without launching Chrome. That claim holds up: the whole interface was constructed, driven and torn down here without a browser existing.

The layout is a vertical stack of panels built with `.pack`, with `.grid` used *inside* each panel for its local rows and columns. Mixing the two managers is fine and idiomatic as long as they never manage the same parent, and here they do not.

Figure 4: The main window, top to bottom. The numbered panel titles ("1.", "2.") encode the required order of operations directly into the interface.

11.2 Styling

A palette of seven colour constants sits at the top of the file, plus a `STATUS_COLORS` dictionary mapping `"info"`, `"success"` and `"error"` to three colours. `set_status` looks the kind up with `.get(kind, STATUS_COLORS["info"])`, so an unrecognised kind degrades to informational rather than raising.

The `ttk` theme is switched to `"clam"` inside a `try`/`except tk.TclError: pass`, because `clam` is not guaranteed present on every platform. That is a correct and cheap piece of defensive programming.

The widget toolkits are mixed inconsistently

Labels, frames and comboboxes are `ttk`; the buttons are plain `tk`. The reason is visible in the code: `tk.Button` accepts `bg`, `fg` and `bd=0` directly, while a `ttk.Button` would need a configured style. The consequence is that hover behaviour has to be hand-rolled, which is what `on_enter_button` and `on_leave_button` are: manual `<Enter>`/`<Leave>` bindings that swap the background colour, work a `ttk` style would do declaratively via `style.map`. And the file *does* use `style.map` correctly, for `Custom.TMenubutton`, a few lines away. Both approaches are present in the same function.

Note also that `highlight_button` never gets those hover bindings, and it would be a bug if it did: the handlers hardcode `ACCENT_COLOR` as the resting colour, but that button rests at `FIELD_BG`. Hovering it would permanently change its colour. The omission is correct; whether

it was reasoned or accidental cannot be told from the source.

11.3 `add_to_history`: a small function done well

```
def add_to_history(combobox, value):
    if not value:
        return
    values = list(combobox["values"])
    if value in values:
        values.remove(value)
    values.insert(0, value)
    combobox["values"] = values[:10]
```

Listing 8: add_to_history

This implements a most-recently-used list: remove the value if already present, push it to the front, cap at 10. Re-entering an old URL moves it back to the top rather than duplicating it. It takes the combobox as a parameter, so unlike most of this file it is reusable, and both fields do reuse it. Verified: after loading one URL the field's dropdown contained exactly that URL.

The history is in memory only and dies with the process, which the README lists as a known limitation rather than hiding.

11.4 `dropdown_var_detect` and its load-bearing guard

```
def dropdown_var_detect(*args):
    if "instructions" not in globals():
        return
    value = dropdown_var.get()
    hints = { "Class": "Enter the class name of the element you want to find", ... }
    instructions.config(text=hints.get(value, "Pick a locator strategy above"))
```

Listing 9: The trace callback, abridged

Jargon: Variable tracing

`dropdown_var.trace("w", dropdown_var_detect)` registers a callback that Tkinter fires every time the `StringVar` is *written*. It is how the instructions label stays in sync with the dropdown with no polling and no button.

The guard that looks like dead code and is not

`if "instructions" not in globals(): return` looks like leftover paranoia, since `instructions` is created in `build_ui`. Tracing the actual order of execution shows it is essential. Inside `build_ui`, the trace is registered, then the `OptionMenu` is constructed with its initial value `"Select"`, and constructing it *writes* the `StringVar`, which fires the callback *immediately*. At that instant the line that creates the `instructions` label is still 50 lines further down and has not run. Without the guard, the very first callback would raise `NameError` during startup, every single time.

This is the best example in the codebase of why reading a function in isolation misleads you. It is also, notably, the one subtlety the README already documents, because a previous pass over this code found the same thing.

The hint dictionary has an entry per strategy and a `.get` default of "Pick a locator strategy above", which is what the initial `"Select"` value falls through to. Verified: setting the variable to `"Class"` put "Enter the class name of the element you want to find" into the label.

12 Shared state: the globals

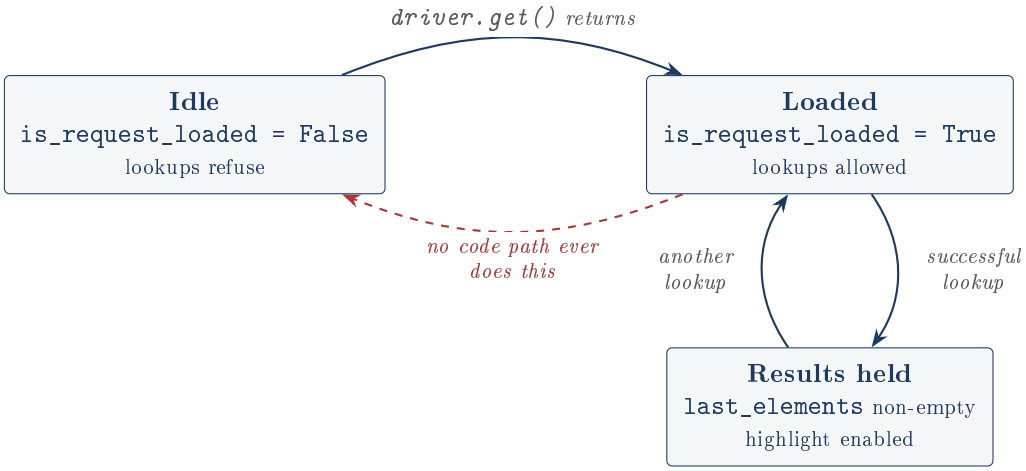

Figure 5: The application's state. The red dashed arrow is the transition that *ought* to exist and does not: no code path ever sets `is_request_loaded` back to `False`. Once true, always true, for the life of the process.

Three module-level globals carry all state:

- `driver`: the live Chrome session. Read by six functions.
- `is_request_loaded`: the guard that stops lookups before a page exists.
- `last_elements`: the most recent match list, for the highlight button.

Jargon: Global variable, and why `global` appears

A **global** here means a variable at module level, shared by every function in the file. Python lets any function *read* one without ceremony, but *assigning* to a name inside a function makes it local unless you declare `global name` first. That is why `load_html` says `global is_request_loaded` (it assigns) and why the lookup functions do too, even though they only read it, which is redundant but harmless.

`driver` is never declared and only exists by luck of ordering

Unlike the other two, `driver` is never given a module-level default. It comes into existence only when `if __name__ == "__main__"` assigns it. Every function that uses it resolves the name at call time, which is why this works at all: by the time any callback fires, the assignment has happened.

The consequence is that `import main; main.load_html()` raises `NameError: name 'driver' is not defined`. The README describes testing against a fake driver, and that is possible, but only by first assigning `main.driver = fake` from outside, which is exactly what had to be done to exercise this code. A module-level `driver = None` would document the contract instead of leaving it implicit.

The globals are the project's real architectural debt

The README states this itself and it is worth restating precisely. Because seven lookup functions read `driver` from module scope rather than receiving it, none of them can be tested without monkey-patching the module, and the application structurally cannot hold two browser sessions at once. A small `Scraper` class holding `driver` and `is_request_loaded` as

instance attributes would fix both and is the change the owner should be ready to describe. The counter-argument, which is legitimate: this is a 551-line single-window tool where the browser genuinely is a singleton, and globals in a Tkinter script are a very well-trodden pattern. The honest framing is not "this is wrong" but "this traded testability for brevity, knowingly".

`build_ui` declares a global it never assigns

The `global` statement in `build_ui` lists `title` among the names it exports. Nothing in the file ever assigns or reads `title`. It is dead, left over from an earlier version, and harmless, but it is the kind of detail an interviewer who reads the code will notice.

13 Dependencies, configuration and licence

13.1 One dependency

`requirements.txt` is a single line: `selenium>=4.15,<5`.

Why that version range is right

The lower bound is not arbitrary. Selenium Manager, the feature that removes the hardcoded driver path, arrived in 4.6, so anything older would break the project's portability story. The upper bound `<5` excludes a future major version, because by convention a major version bump is allowed to break the API. Pinning a floor for the feature you need and a ceiling below the next breaking change is exactly the correct shape for a dependency range.

Everything else is standard library: `os` for environment variables, `tkinter` and `tkinter.ttk` for the interface. `tkinter` ships with CPython on Windows and macOS but is a separate package on Debian and Ubuntu (`python3-tk`), which the README correctly warns about.

13.2 Configuration and secrets

`.env.example` documents both variables and states that both are optional and the app runs with neither set. Neither is a secret: one is a filesystem path, the other a boolean. There is no API key, no credential, and nothing to leak. `.gitignore` nonetheless excludes `.env`, which is the right default.

Note that nothing in `main.py` actually reads a `.env` file: there is no `python-dotenv` dependency. The variables must be real environment variables. `.env.example` is documentation of the names, not an input the program parses.

13.3 Licence

`LICENSE` is PolyForm Strict 1.0.0: the source is public to read, but reuse, modification and redistribution are not granted. For a portfolio piece meant to be inspected rather than adopted, that is a coherent choice.

14 Honest findings, collected

Everything below was found by reading the source and, where possible, by running it. Nothing here is speculation unless it says so.

14.1 Defects in the code

1. `is_request_loaded` **is set before the load is confirmed**, so a page-load timeout leaves the flag wrongly `True`.
2. **Only** `InvalidArgumentException` **is caught around** `driver.get`, so a dead host or crashed browser produces an uncaught traceback and a frozen-looking status bar.
3. **The zero-match code path is unreachable**, so "no matches" is reported as "please enter a valid class name", blaming the user for a valid locator.
4. `last_elements` **and the highlight button are never reset** on navigation or on a failed lookup, so the button stays live pointing at a previous page's elements.
5. **Every lookup opens a new** `Toplevel` and none are ever closed.
6. `driver` **has no module-level definition**, so importing the module and calling anything raises `NameError` until it is patched in.
7. `title` **is declared global and never used.**
8. `maximize_window()` **runs in the headless path** where it is meaningless.
9. `dropdown_var.trace("w", ...)` is the legacy Tk API. The modern spelling is `trace_add("write", ...)`. Verified working without a deprecation warning on Python 3.12.3 here, so this is a style point today and a maintenance risk later, not a present bug.

14.2 Claims in the README that the code does not support

The instruction is that where documentation and code disagree, the code wins. Three disagreements were found, and all three are the README overstating.

The README describes a test suite that is not in the repository

The "What was verified here" section claims a "standalone unit test suite" of 7 passing cases covering the formatting functions, and a second test that builds the window against a live X display and drives it with a fake driver. It then says "see `Test plan` below for the exact command and output".

There is no `Test plan` section in the README. There is no test file anywhere in the repository: `git ls-files` lists eight files and none of them is a test. `git log` across all history shows a test file was never committed and later removed; it was never there.

The underlying claims do appear to be *true*. Equivalent tests were written from scratch for this document and every stated behaviour held: the formatting functions handle truncation, whitespace, missing attributes and total staleness correctly, and the full window does construct and drive against a fake driver on a real display. So the honest reading is that the tests were written, run, and then not committed, and the cross-reference to a section that was never written is the loose thread that exposes it.

This is the most important finding in this document, because an interviewer who reads "all 7 cases pass; see Test plan below" and then looks for either the plan or the tests will find neither. Either commit the tests or remove the claim. Do not leave it as is.

`temp.py` and `temp1.py` have never existed in this repository

The Challenges section says the first working version is "still visible as `temp.py` / `temp1.py` in the source history before this rewrite". It is not. The repository has four commits, beginning with "Initial commit: custom web scraper", and searching every commit for every file ever added (`git log --all --diff-filter=A`) returns eight paths, none named `temp`. The BeautifulSoup draft the sentence refers to may well have existed on the author's machine, but it is

not in this history, and the README invites a reader to go look for something they cannot find.

The results table is described as "sortable" and is not

Covered in full above. `ttk.Treeview` sorts only when you bind a command to each heading, and `main.py` binds none.

"Two layers in one file", followed by three bullet points

The Architecture section opens "Two layers in one file, split by `if __name__ == "__main__":`" and then lists three: Selenium, Tkinter, and pure formatting. A small editing artefact from when the formatting layer was added, but it is the first line of the section an interviewer reads. Also, the split it names is not what separates the layers; the `__main__` guard separates definition from execution.

14.3 What the project gets right

Balance matters, and there is real quality here:

- The `WebDriverWait` plus expected-conditions pattern is the correct tool for the problem, used consistently, and never a `time.sleep`.
- The pure formatting layer is a genuine piece of design: extracting the logic that does not need a browser so it can be tested without one is exactly the right instinct, and the per-field `_safe_call` granularity shows the stale-element problem was actually understood rather than merely caught.
- The environment-variable driver resolution is a real fix to a real portability bug, and the version range in `requirements.txt` is chosen for a reason.
- `update_idletasks` before a blocking call, `rowconfigure` weights for resize, the `clam` theme in a `try`/`except`, the dictionary miss doing double duty as validation: these are the marks of someone who has used the toolkit, not just imported it.
- The README's "What I'd do differently" section is honest about the globals and the duplication without being asked. The gap is in the verification claims, not in the self-assessment.

15 What was verified for this document

Every claim below was produced by running code on this machine, not by reading.

- `python3 -m py_compile main.py` succeeds. Python 3.12.3, selenium 4.45.0, `tkinter` importable.
- The formatting layer was driven with fake duck-typed elements: whitespace collapsing, exact 80-character truncation with ellipsis, attribute filtering and skipping of empty values, 1-based row numbering, and a fully stale element yielding `(3, '?', '', '')`. All behaved as this document describes.
- The complete window was built via `build_ui()` on a real display with `main.driver` patched to a fake and `WebDriverWait` stubbed. The three callbacks were then driven in turn: first `load_html`, then `on_get_elements_click`, then `highlight_in_browser`. They produced, in order: a "Loaded" status naming the test URL; a one-entry URL history; the correct instructions text for "Class"; a "Found 2 elements" status; the highlight button enabled; `last_elements` holding 2 items; a status confirming 2 elements highlighted; and exactly 2 recorded calls to `execute_script`.

- The zero-match branch was reached only by stubbing out `WebDriverWait`, which is the direct evidence that it is unreachable through the real code path.
- The repository's full history was searched for the test file and for `temp.py`/`temp1.py`. Neither has ever been committed.
- `StringVar.trace("w", ...)` was confirmed to run without raising or warning on this Python.

Not verified: a live Chrome session against a real website, and the highlight JavaScript against a real DOM. That needs a Chrome install and outbound network access for Selenium Manager to fetch a driver. The Selenium calls used are standard, stable 4.x API, but "it drives real Chrome correctly" rests on inspection here, not on execution.

16 Closing summary

What to say about this project in one breath

A single-file desktop tool that drives real Chrome through Selenium so you can test locator strategies against a live, JavaScript-rendered page without writing a script per question. Its best idea is extracting the display-formatting logic into pure, duck-typed functions that need no browser to test. Its worst is that everything else shares state through module globals, which is why the other layers cannot be tested the same way. Its most defensible fix is replacing a hardcoded Windows chromedriver path with Selenium Manager plus an environment-variable override. Its most urgent outstanding job is not in the code at all: the README claims a test suite that is not in the repository.